

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en macOS con Apple Silicon (M1/M2/M3/M4), Zsh, Keychain, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en macOS (Apple Silicon M1/M2/M3/M4)

Plataforma: macOS Sonoma / Sequoia / versiones modernas con arquitectura ARM64 (Apple Silicon)

Motor: Unity 2022 LTS / Unity 6 (6000.x) optimizado para Apple Silicon

Herramientas: Zsh, Git 2.40+, Git LFS, GitHub CLI (`gh`), GitHub Desktop, Apple Keychain, Homebrew en `/opt/homebrew`, Unity Hub, UnityYAMLMerge

Índice de Contenidos

1. [Parte I: Fundamentos y Preparación del Entorno Unity en Apple Silicon](#)
 - 1.1 [Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora](#)
 - 1.2 [Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Homebrew](#)
 - 1.3 [Configuración Crítica del Editor: Visible Meta Files y Force Text](#)
 - 1.4 [La Regla de Oro de los Archivos `.meta` y los GUIDs](#)
 - 1.5 [El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity](#)
 - 1.6 [Configuración Exhaustiva de Git LFS con `.gitattributes`](#)
2. [Parte II: Flujo de Trabajo Esencial Diario \(Nivel Novato\)](#)
 - 2.1 [Inicializar y Publicar un Proyecto en GitHub \(Terminal Zsh y GitHub Desktop\)](#)
 - 2.2 [Clonación Correcta de Proyectos con Git LFS en macOS](#)
 - 2.3 [El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits](#)
 - 2.4 [Conventional Commits Aplicados al Desarrollo de Videojuegos](#)
 - 2.5 [Sincronización sin Romper la Cache \(`pull --rebase` vs Fetch en Desktop\)](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Estrategia de Ramas en Equipos de Videojuegos \(Zsh y Desktop\)](#)
 - 3.2 [Arquitectura de Escenas Divididas \(Multi-Scene Editing Aditivo\)](#)
 - 3.3 [Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes](#)
 - 3.4 [Configuración de UnityYAMLMerge en macOS como Mergetool](#)
 - 3.5 [Uso de Git Stash, Historial y Reversión Segura \(Zsh y Desktop\)](#)
4. [Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente](#)
 - 4.1 [Tema 1: Prevención Arquitectónica de Conflictos en Unity](#)
 - 4.2 [Tema 2: Conflictos en Archivos `.meta` \(GUID Desincronizado\)](#)
 - 4.3 [Tema 3: Conflictos en Scripts C# \(`.cs` \)](#)
 - 4.4 [Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge \(Zsh y Desktop\)](#)

- 4.5 [Tema 5: Forzar una Versión Completa de Asset \(`--ours` vs `--theirs` \)](#)
 - 4.6 [Tema 6: Cambios Locales en el Editor al Hacer Pull](#)
 - 4.7 [Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS](#)
 - 4.8 [Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock](#)
 - 4.9 [Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano](#)
5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
- 5.1 [Git Worktrees en macOS: Múltiples Ramas sin Recargar `Library/`](#)
 - 5.2 [Depuración Binaria con Git Bisect y Blame en C#](#)
 - 5.3 [GitHub Codespaces desde la Terminal de Mac](#)
 - 5.4 [GitHub Copilot CLI en Zsh](#)
 - 5.5 [Git Hooks y Validación Pre-commit de Archivos `.meta` en macOS](#)
6. [Parte VI: Gestión de Paquetes UPM y Dependencias](#)
- 6.1 [Instalación de Paquetes Mediante URLs de Git en UPM](#)
 - 6.2 [Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens](#)
7. [Parte VII: Automatización CI/CD con GitHub Actions y GameCI](#)
- 7.1 [Arquitectura de GameCI para macOS Standalone \(Apple Silicon\)](#)
 - 7.2 [Activación de Licencias de Unity en GitHub Actions](#)
 - 7.3 [Pipeline Automatizado: Tests y Compilación StandaloneOSX \(.app / .dmg\)](#)
 - 7.4 [Subida Automática de Builds a GitHub Releases](#)
8. [Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos](#)
- 8.1 [Branch Protection Rules y Rulesets](#)
 - 8.2 [Gestión de Secretos para APIs de Juegos \(Apple Game Center, Steam\)](#)
 - 8.3 [Gobernanza con `CODEOWNERS` para Artistas y Programadores](#)
9. [Parte IX: Distribución y Despliegue con GitHub Releases](#)
- 9.1 [Creación Automatizada de Releases con Tags Semánticos](#)
 - 9.2 [Empaquetado y Publicación de Archivos DMG y Aplicaciones macOS](#)
10. [Parte X: Catálogo Maestro de Incidentes Críticos de Unity en macOS](#)
- 10.1 [Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs](#)
 - 10.2 [Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto](#)
 - 10.3 [Incidente 3: Subida Accidental de la Carpeta `Library/` o Archivos `.DS\_Store`](#)
 - 10.4 [Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB](#)
 - 10.5 [Incidente 5: Shaders Magenta / Rosados tras Clonar en macOS](#)
 - 10.6 [Incidente 6: Límite de Almacenamiento y Ancho de Banda de Git LFS Superado](#)
 - 10.7 [Incidente 7: Desfase de Versiones Menores del Editor de Unity](#)

- 10.8 [Incidente 8: Archivos Bloqueados por Procesos de Unity en Ejecución](#)
 - 10.9 [Incidente 9: Fuga de Claves Privadas en ScriptableObjects o Certificados Apple](#)
 - 10.10 [Incidente 10: Regeneración Limpia y Segura de la Caché Local en macOS](#)
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Apple Silicon

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

En macOS con chips Apple Silicon (M1/M2/M3/M4), Unity compila assets a gran velocidad gracias a la memoria unificada. Sin embargo, la estructura interna debe respetarse estrictamente para evitar subir cachés innecesarias a Git:

```
PLAINTEXT

MiVideojuegoMac/
├─ Assets/           → [OBLIGATORIO EN GIT] Scripts C#, Escenas, Prefabs, Texturas,
Modelos 3D y .meta.
├─ Packages/         → [OBLIGATORIO EN GIT] manifest.json y packages-lock.json (UPM).
├─ ProjectSettings/  → [OBLIGATORIO EN GIT] Configuración del juego (Graphics Metal,
Input, Physics).
├─ Library/          → [¡NUNCA EN GIT!] Cache de assets compilada para la GPU Metal
local.
├─ Temp/             → [¡NUNCA EN GIT!] Archivos volátiles de compilación del editor.
├─ Obj/ & Build/      → [¡NUNCA EN GIT!] Binarios compilados y bundles `.app`.
├─ .DS_Store         → [¡NUNCA EN GIT!] Metadatos de visualización de carpetas del
Finder de macOS.
├─ UserSettings/     → [¡NUNCA EN GIT!] Preferencias de ventanas del usuario en macOS.
└─ Logs/             → [¡NUNCA EN GIT!] Registros de compilación del motor.
```

1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Homebrew

En macOS con Apple Silicon, Homebrew se instala en `/opt/homebrew`. Ejecuta en la terminal **Zsh**:

```
# 1. Instalar Git nativo ARM64, Git LFS, GitHub CLI y GitHub Desktop
brew install git git-lfs gh
brew install --cask github

# 2. Inicializar los filtros de Git LFS
git lfs install

# 3. Configurar integración con el Llavero de macOS (Apple Keychain)
git config --global credential.helper osxkeychain
```

¿Qué hace este comando?

Instala las versiones optimizadas para ARM64 de Git, Git LFS y GitHub CLI, instala la aplicación gráfica oficial de **GitHub Desktop** para macOS, y activa `osxkeychain` para que macOS gestione de forma segura y transparente las credenciales de GitHub con Touch ID.

1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text

Antes de inicializar Git, verifica estos parámetros en el Editor de Unity:

1. Abre tu proyecto en Unity en macOS.
2. Navega al menú superior: **Unity** → **Project Settings** → **Version Control**.
 - En **Mode**, selecciona estrictamente: `Visible Meta Files`.
3. Navega a **Unity** → **Project Settings** → **Editor**.
 - En **Asset Serialization Mode**, selecciona estrictamente: `Force Text`.

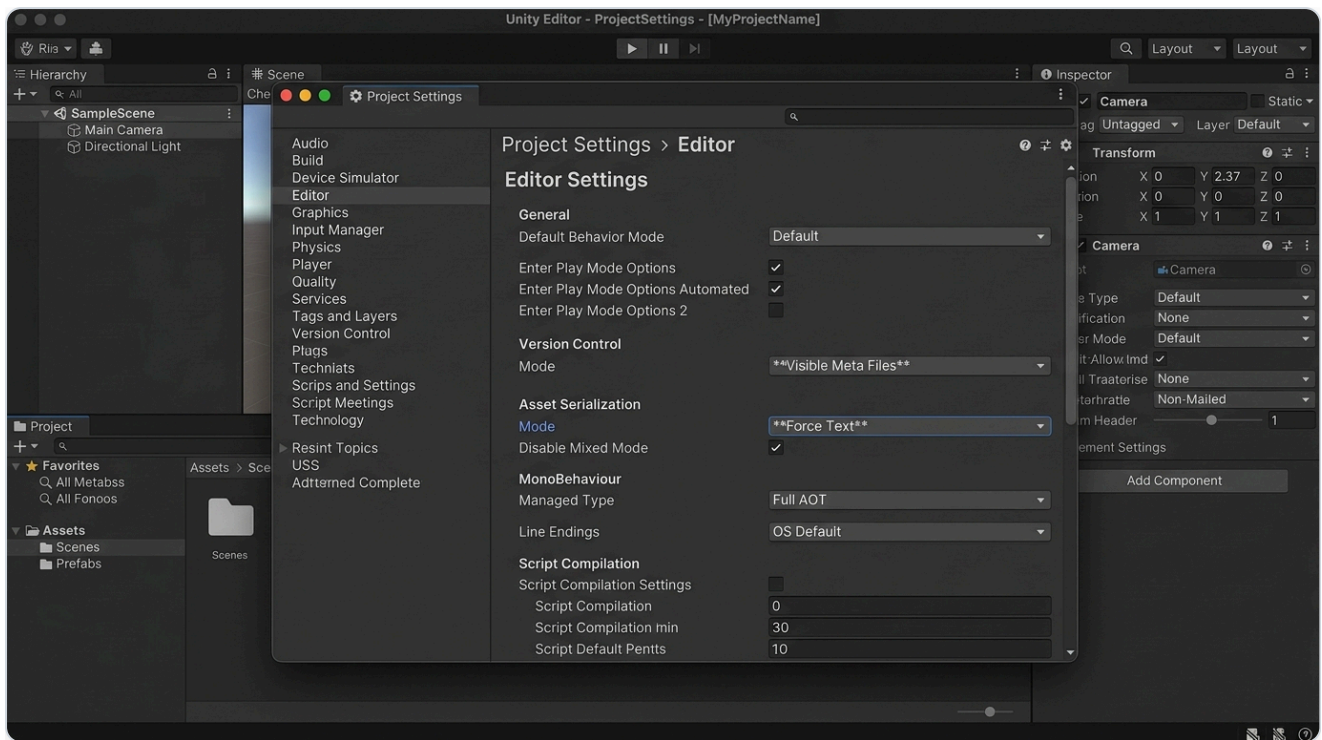


Figura 1.1: Configuración en Unity Project Settings (Editor) en macOS estableciendo Visible Meta Files y Force Text para serialización YAML.

IMPORTANT

Force Text obliga al motor a serializar escenas (`.unity`), prefabs (`.prefab`) y materiales (`.mat`) en formato de texto plano **YAML**, permitiendo a Git generar diffs legibles y resolver fusiones.

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada archivo en la carpeta `Assets/` está emparejado con un `.meta` que contiene su GUID único:

```
fileFormatVersion: 2
guid: a7c12f80e94b4d6b8890123456789abc
```

CAUTION

LA REGLA FUNDAMENTAL:

Si renombras o mueves un asset desde el Finder de macOS o desde Zsh, **debes renombrar o mover su archivo `.meta` exactamente igual**. Nunca hagas commit de un asset sin su respectivo `.meta` acompañante para evitar la pérdida de referencias ("Missing Script").

1.5 El Archivo `.gitignore` Oficial y Optimizado para macOS y Unity

Crea `.gitignore` en la raíz del repositorio:

```
# =====  
# .gitignore para Unity en macOS (Apple Silicon)  
# =====  
  
[Ll]ibrary/  
[Tt]emp/  
[Oo]bj/  
[Bb]uild/  
[Bb]uilds/  
[Ll]ogs/  
[Uu]ser[Ss]ettings/  
[Mm]emoryCaptures/  
  
# Asset server cache  
sysinfo.txt  
*.stackdump  
  
# Entornos de desarrollo macOS (Xcode / Rider / VS Code)  
.vs/  
.idea/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.svd  
*.pdb  
*.opendb  
*.VC.db  
  
# Compilaciones de macOS (.app / .dmg) y paquetes  
*.app  
*.dmg  
*.apk  
*.unitypackage  
  
# Archivos específicos de macOS  
.DS_Store
```

```
.AppleDouble  
.LSOverride  
._*  
.DocumentRevisions-V100  
.fsevents  
.Spotlight-V100  
.TemporaryItems  
.Trashes
```

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Guarda este archivo `.gitattributes` en la raíz del proyecto:

```
# =====  
# Configuración Git LFS para Unity en macOS  
# =====  
  
# Formatos 3D y Rigs  
*.fbx filter=lfs diff=lfs merge=lfs -text  
*.obj filter=lfs diff=lfs merge=lfs -text  
*.blend filter=lfs diff=lfs merge=lfs -text  
*.dae filter=lfs diff=lfs merge=lfs -text  
*.max filter=lfs diff=lfs merge=lfs -text  
  
# Texturas e Imágenes de Alta Resolución  
*.psd filter=lfs diff=lfs merge=lfs -text  
*.tga filter=lfs diff=lfs merge=lfs -text  
*.png filter=lfs diff=lfs merge=lfs -text  
*.jpg filter=lfs diff=lfs merge=lfs -text  
*.exr filter=lfs diff=lfs merge=lfs -text  
*.hdr filter=lfs diff=lfs merge=lfs -text  
*.tif filter=lfs diff=lfs merge=lfs -text  
  
# Audio y Vídeo  
*.wav filter=lfs diff=lfs merge=lfs -text  
*.mp3 filter=lfs diff=lfs merge=lfs -text  
*.ogg filter=lfs diff=lfs merge=lfs -text  
*.mp4 filter=lfs diff=lfs merge=lfs -text  
*.mov filter=lfs diff=lfs merge=lfs -text  
  
# Bloqueo Concurrente para Binarios  
*.fbx lockable  
*.psd lockable  
*.blend lockable  
*.wav lockable  
  
# Fusión Semántica de Unity y Finales de Línea LF  
*.cs text diff=csharp eol=lf  
*.json text eol=lf  
*.shader text eol=lf  
*.unity merge=unityyamlmerge eol=lf  
*.prefab merge=unityyamlmerge eol=lf  
*.mat merge=unityyamlmerge eol=lf
```

```
*.asset merge=unityyamlmerge eol=lf  
*.meta text eol=lf
```

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto en GitHub (Terminal Zsh y GitHub Desktop)

Modalidad A: Vía Terminal Zsh



```
cd ~/UnityProjects/MiVideojuego
git init
git lfs install
git add .gitattributes .gitignore
git commit -m "chore: inicializar configuracion Unity con LFS y gitignore"
git branch -M main
gh repo create MiVideojuegoMac --private --source=. --remote=origin --push
```

¿Qué hace este comando?

Inicializa el repositorio local, activa Git LFS, crea el commit fundacional y publica el repositorio como privado en GitHub vinculando la rama `main`.

Modalidad B: Vía GitHub Desktop en macOS

1. Abre **GitHub Desktop** desde el Launchpad o Spotlight (`Cmd + Espacio`).
2. Pulsa en **File** → **Add Local Repository** (o `Cmd + O`).
3. Selecciona la carpeta de tu juego en tu Mac.
4. Si la carpeta aún no tiene repositorio, pulsa en el enlace azul **"create a repository"**.
5. Pulsa en el botón azul **Add Repository**.
6. En la barra superior, pulsa en **Publish repository** para crearlo en tu cuenta de GitHub.

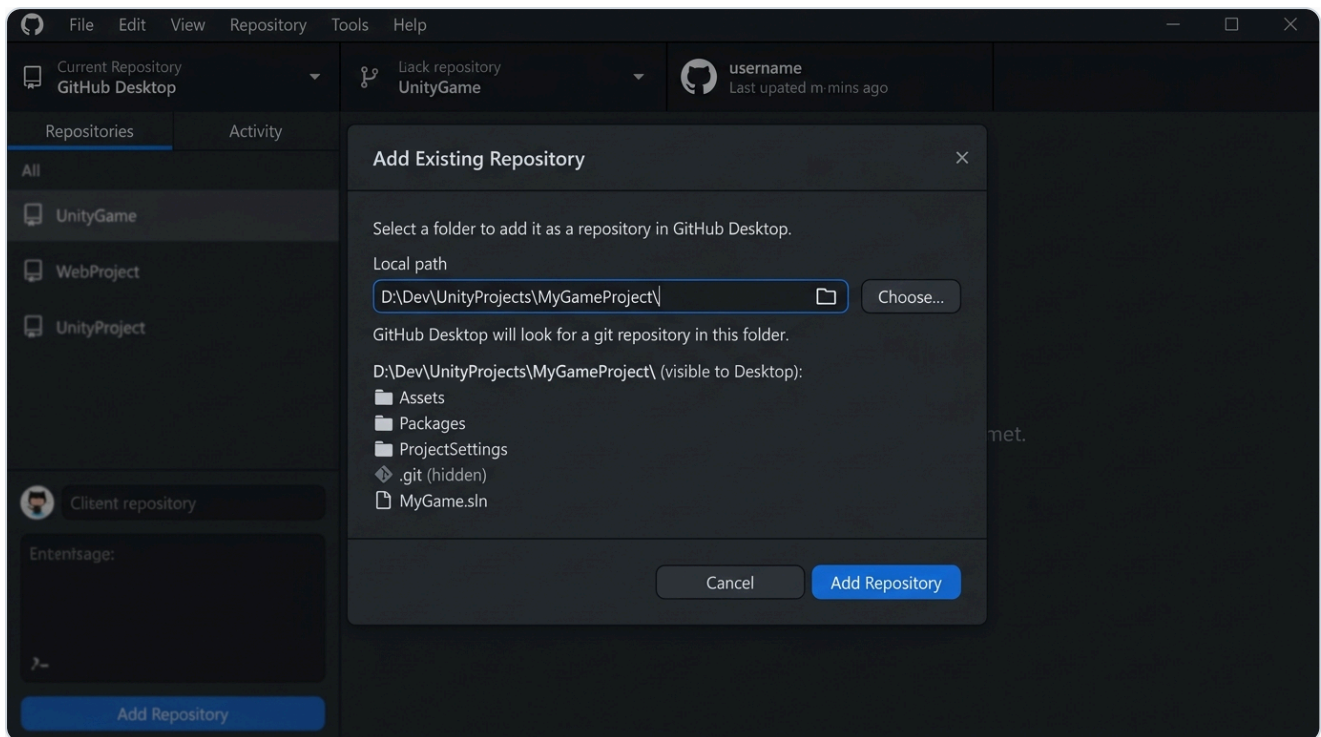


Figura 2.1: Cuadro de diálogo 'Add Existing Repository' en GitHub Desktop en macOS vinculando la carpeta del proyecto Unity.

2.2 Clonación Correcta de Proyectos con Git LFS en macOS

Modalidad A: Vía Terminal Zsh

```
git lfs install
git clone git@github.com:mi-organizacion/videojuego.git
cd videojuego
git lfs pull
```

Modalidad B: Vía GitHub Desktop

1. Pulsa en **File** → **Clone Repository** (o `Cmd + Shift + O`).
2. Selecciona el repositorio de la lista o escribe la URL.
3. Elige la ruta de destino local en tu Mac y pulsa **Clone**.
4. GitHub Desktop gestionará la descarga completa de los assets pesados de LFS automáticamente.

2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits

Modalidad A: Vía Terminal Zsh



```
# 1. Comprobar archivos modificados
git status

# 2. Agregar scripts y prefabs asegurando sus respectivos .meta
git add Assets/Scripts/PlayerController.cs Assets/Scripts/PlayerController.cs.meta
git add Assets/Prefabs/Player.prefab Assets/Prefabs/Player.prefab.meta

# 3. Confirmar commit
git commit -m "feat(player): implementar salto con fisica Rigidbody2D"
```

Modalidad B: Vía GitHub Desktop

1. En la pestaña **Changes** de la barra lateral izquierda, inspecciona los archivos modificados.
2. Comprueba que cada script o prefab tenga marcada su casilla junto a su archivo `.meta`.
3. En el panel derecho, verifica el diff de cambios con colores verde (añadido) y rojo (eliminado).
4. Introduce el título en **Summary** y contexto adicional en **Description**.
5. Pulsa en el botón azul **Commit to main**.

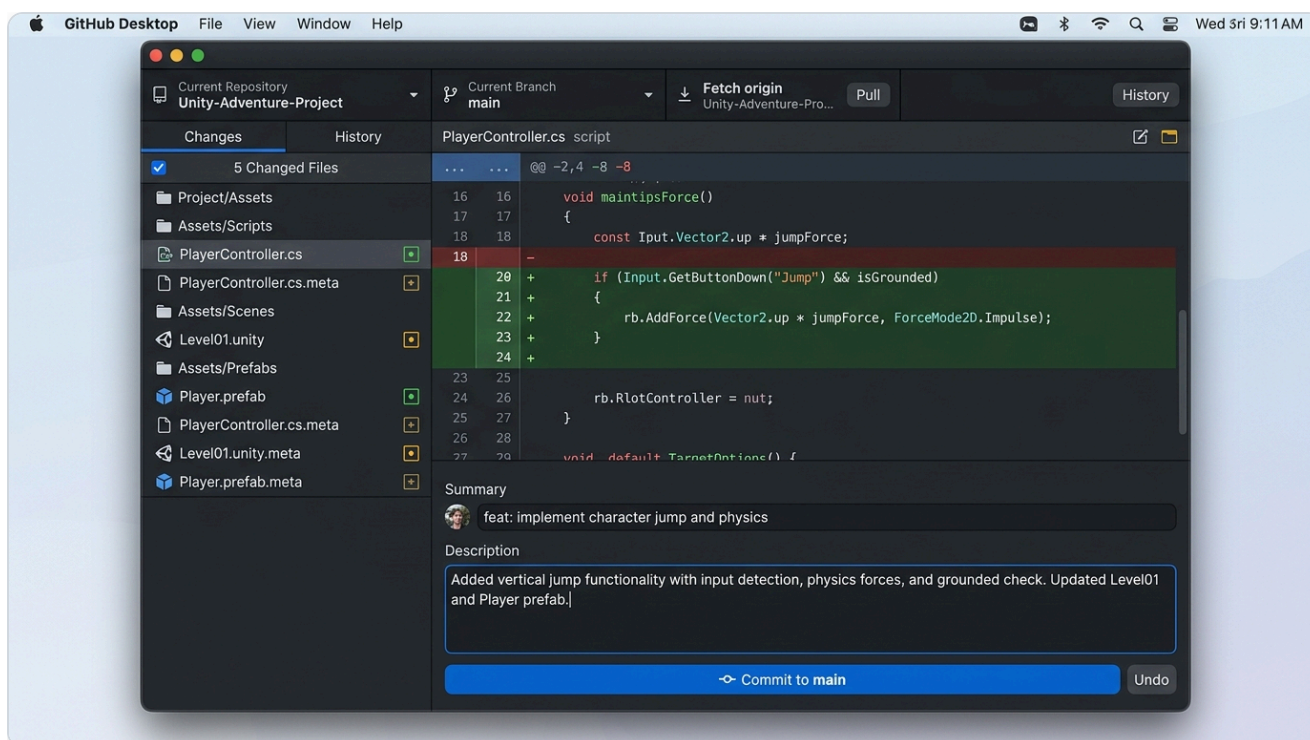


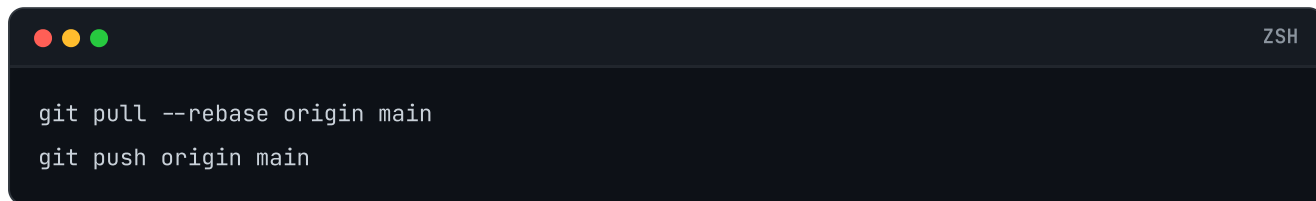
Figura 2.2: Pestaña de cambios en GitHub Desktop verificando en macOS la consistencia de archivos y metadatos de Unity.

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

Prefijo	Área de Aplicación en Videojuegos	Ejemplo
feat:	Nuevas mecánicas o sistemas jugables	feat(player): anadir aceleracion y derrape en curvas
fix:	Corrección de fallos o glitches visuales	fix(metal): corregir artefacto de reflejo en shaders Metal
art:	Modelos 3D, texturas y animaciones	art(hero): importar animacion de correr y salto doble
level:	Modificaciones de escenas y diseño de niveles	level(volcano): anadir zonas de lava con dano continuo
audio:	Efectos de sonido y mezclas de AudioManager	audio(fx): balancear volumen de pasos en superficies metalicas
perf:	Optimizaciones de draw calls y memoria	perf(shaders): optimizar shaders URP para GPUs Apple Silicon
chore:	Mantenimiento de paquetes UPM o gitignore	chore(upm): actualizar Unity Collections a version 2.1.4

2.5 Sincronización sin Romper la Cache (`pull --rebase` vs Fetch en Desktop)

Modalidad A: Vía Terminal Zsh

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar on the right says "ZSH". The terminal contains two lines of text: `git pull --rebase origin main` and `git push origin main`.

```
git pull --rebase origin main
git push origin main
```

El rebase mantiene la historia lineal, previniendo que Unity invalide cachés de compilación locales al recibir cambios.

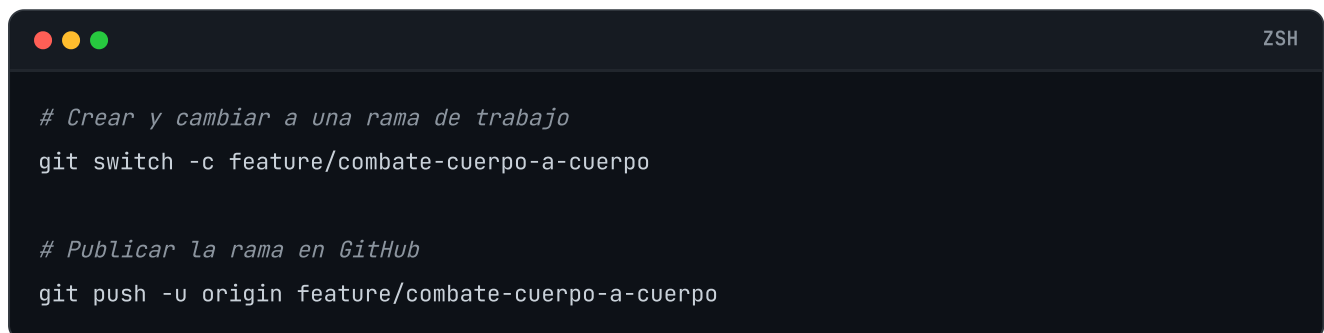
Modalidad B: Vía GitHub Desktop

1. Pulsa en **Fetch origin** en la barra superior.
 2. Si hay commits nuevos, pulsa en el botón **Pull origin**.
 3. Envía tus cambios pendientes haciendo clic en **Push origin**.
-

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos (Zsh y Desktop)

Modalidad A: Vía Terminal Zsh



```
# Crear y cambiar a una rama de trabajo
git switch -c feature/combate-cuerpo-a-cuerpo

# Publicar la rama en GitHub
git push -u origin feature/combate-cuerpo-a-cuerpo
```

Modalidad B: Vía GitHub Desktop

1. En la barra superior, haz clic en el menú **Current Branch**.
2. Escribe el nombre de la nueva rama (ej. `feature/combate-cuerpo-a-cuerpo`).
3. Haz clic en el botón azul **New branch...**
4. Haz clic en **Publish branch** para sincronizarla con GitHub.

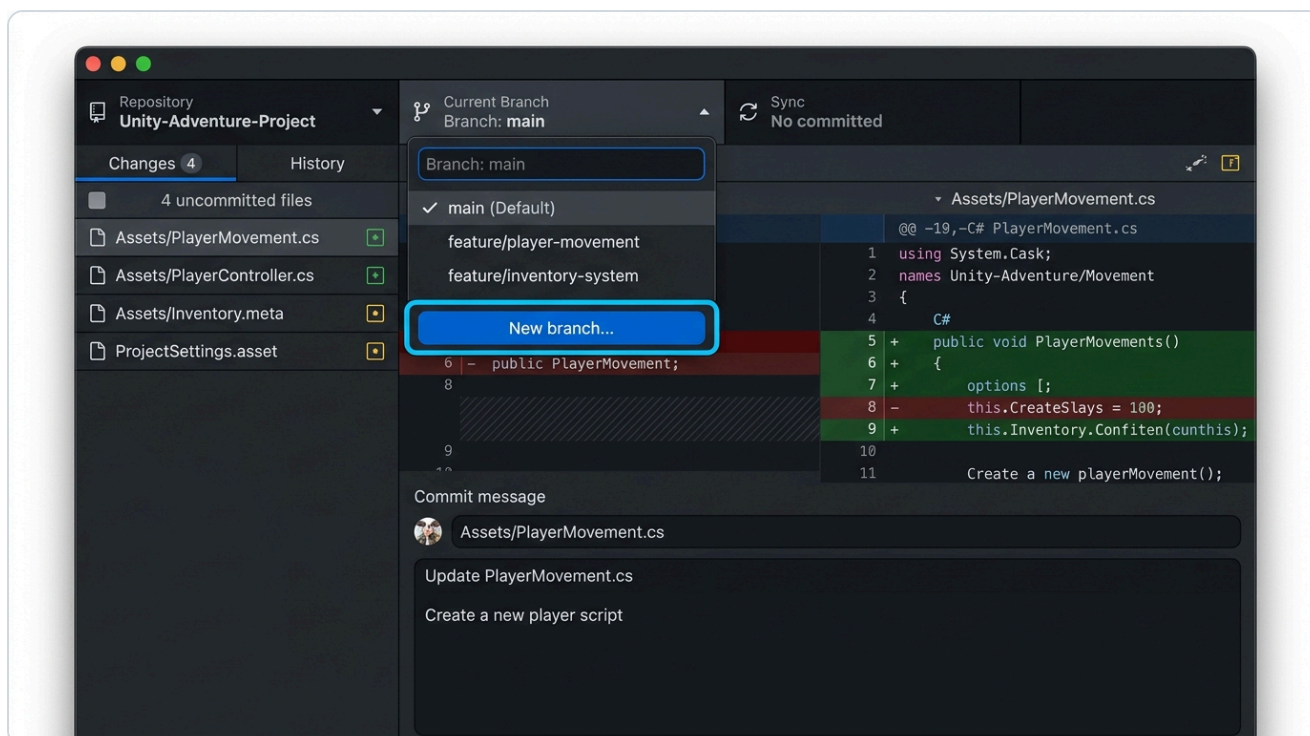


Figura 3.1: Menú desplegable 'Current Branch' en GitHub Desktop facilitando la creación y conmutación de ramas de trabajo en macOS.

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

Para evitar que artistas y programadores choquen al editar la misma escena, subdivide el mundo en subescenas:

```

Nivel_Montaña/
├─ Nivel_Core.unity      → Cámaras, GameManagers, UI Canvas.
├─ Nivel_Entorno.unity   → Terreno, mallas y vegetación (Artistas).
├─ Nivel_Iluminacion.unity → Luces y Post-Processing (Iluminadores).
└─ Nivel_Gameplay.unity  → Enemigos, checkpoints y triggers (Diseñadores).
  
```

Script C# para Carga Aditiva en Tiempo de Ejecución:

Guarda en `Assets/Scripts/AdditiveLoader.cs` :

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class AdditiveLoader : MonoBehaviour
{
    [SerializeField] private string[] additiveScenes = {
        "Nivel_Entorno",
        "Nivel_Iluminacion",
        "Nivel_Gameplay"
    };

    private void Start()
    {
        foreach (string scene in additiveScenes)
        {
            if (!SceneManager.GetSceneByName(scene).isLoaded)
            {
                SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive);
            }
        }
    }
}
```

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

- Trabaja siempre dentro del **Prefab Mode** de Unity.
- Usa **Prefab Variants** para variantes de enemigos o props. Las variantes solo guardan las diferencias (*overrides*), evitando conflictos de fusión en el archivo base.

3.4 Configuración de UnityYAMLMerge en macOS como Mergetool

En macOS, `UnityYAMLMerge` se encuentra dentro del paquete `.app` del editor:

```
# Detectar la versión de Unity instalada más reciente en /Applications/Unity/Hub/Editor/
UNITY_VERSION=$(ls /Applications/Unity/Hub/Editor | tail -n 1)
MERGE_TOOL="/Applications/Unity/Hub/Editor/${UNITY_VERSION}/Unity.app/Contents/Tools/UnityYAMLMerge"

# Configurar en Git
git config --global merge.unityyamlmerge.name "Unity Smart Merge"
git config --global merge.unityyamlmerge.driver "\"${MERGE_TOOL}\" merge -h -p -- '%0' '%B' '%A' '%A'"
git config --global merge.unityyamlmerge.trustExitCode true
git config --global merge.unityyamlmerge.recursive binary
```

¿Qué hace este comando?

Localiza la herramienta nativa `UnityYAMLMerge` en macOS y la configura en Git como motor de fusión tridireccional para resolver conflictos semánticos en escenas y prefabs.

3.5 Uso de Git Stash, Historial y Reversión Segura (Zsh y Desktop)

Modalidad A: Vía Terminal Zsh

```
# Guardar cambios sin confirmar
git stash save "WIP: ajuste de shaders Metal"

# Cambiar de rama y revisar bug
git switch main

# Restaurar cambios
git switch feature/mi-rama
git stash pop
```

Modalidad B: Vía GitHub Desktop

1. Al conmutar de rama con cambios sin confirmar, pulsa en **"Leave my changes on [rama-actual]"** (crea un stash).
2. Para restaurar, pulsa en **Restore** dentro de **Stashed Changes** en la barra lateral.
3. Para deshacer un commit defectuoso: ve a la pestaña **History**, haz clic derecho sobre el commit → **Revert changes in commit**.

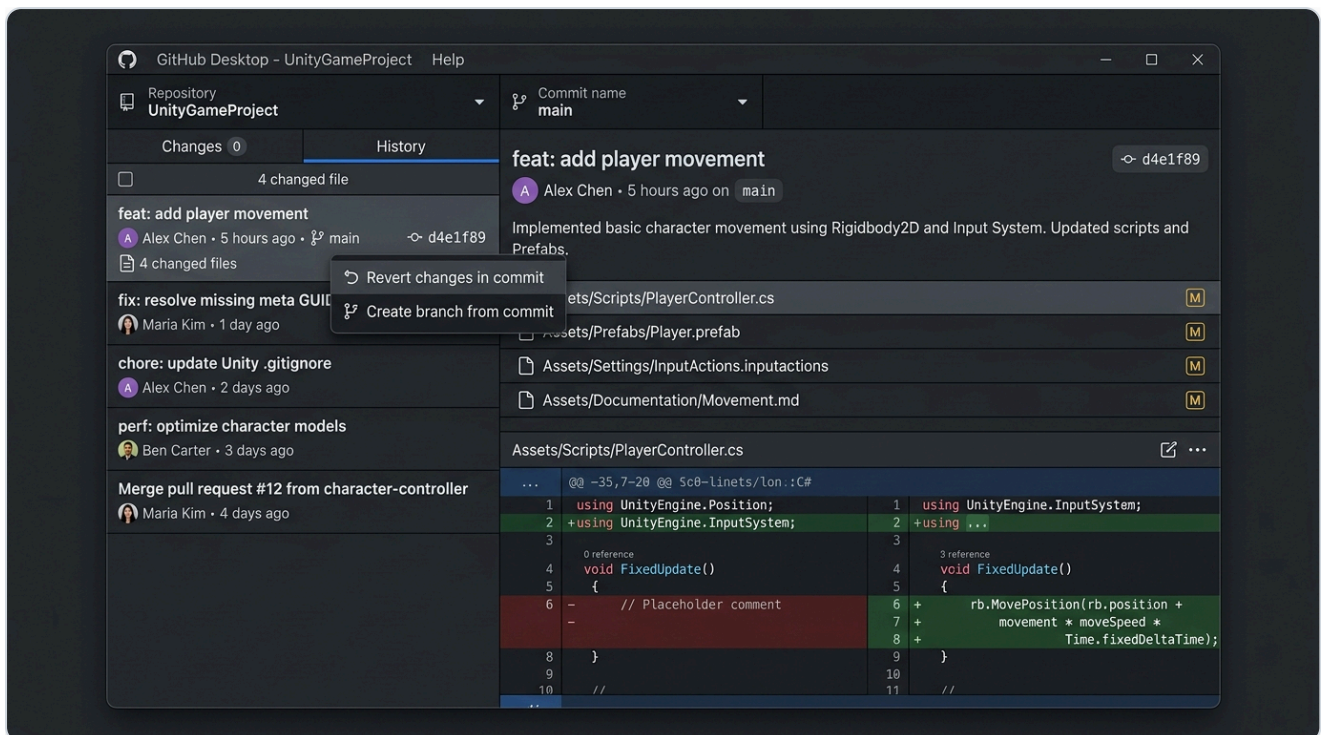


Figura 3.2: Pestaña 'History' en GitHub Desktop en macOS permitiendo inspección de diffs y reversión limpia de commits problemáticos.

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

- Organiza las carpetas por subsistemas funcionales.
 - Emplea escenas aditivas para distribuir responsabilidades por perfiles técnicos.
 - Utiliza prefabs independientes para cada entidad interactiva.
-

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)



```
# Ver archivos en conflicto
git status
```

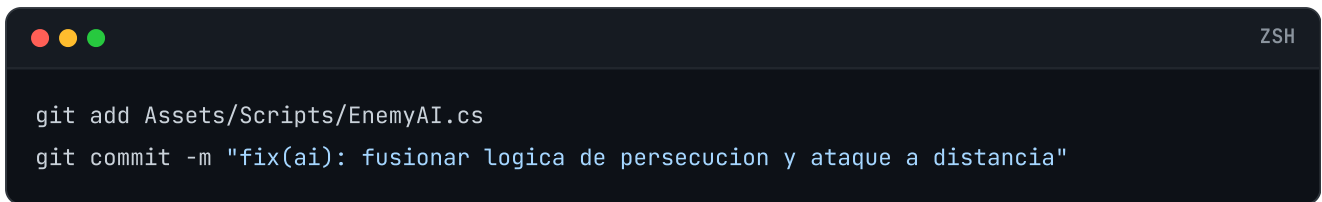
1. Abre el archivo `.meta` en VS Code o tu editor preferido.
2. Si el archivo ya estaba referenciado en escenas por otro desarrollador, conserva el GUID del servidor remoto.
3. Elimina los marcadores de conflicto dejando una sola línea `guid: .`
4. Confirma la resolución:



```
git add Assets/Textures/Heroe.png.meta
git commit -m "fix(meta): resolver colision de GUID en textura de Heroe"
```

4.3 Tema 3: Conflictos en Scripts C# (`.cs`)

Combina las líneas en VS Code o JetBrains Rider, guarda y ejecuta:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal contains two lines of text: `git add Assets/Scripts/EnemyAI.cs` and `git commit -m "fix(ai): fusionar logica de persecucion y ataque a distancia"`.

```
git add Assets/Scripts/EnemyAI.cs
git commit -m "fix(ai): fusionar logica de persecucion y ataque a distancia"
```

4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (Zsh y Desktop)

Modalidad A: Vía Terminal Zsh

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner says "ZSH". The terminal contains one line of text: `git mergetool -t unityyamlmerge`.

```
git mergetool -t unityyamlmerge
```

UnityYAMLMerge resolverá de forma semántica los IDs internos de la escena sin corromper el formato YAML.

Modalidad B: Vía GitHub Desktop

1. Tras un merge con colisión, aparece la ventana modal **Resolve conflicts before merging**.
2. En la fila del archivo de escena (`.unity`), haz clic en **Open in UnityYAMLMerge**.
3. Tras la fusión automática, haz clic en el botón azul **Resolve Conflicts** para sellar el merge.

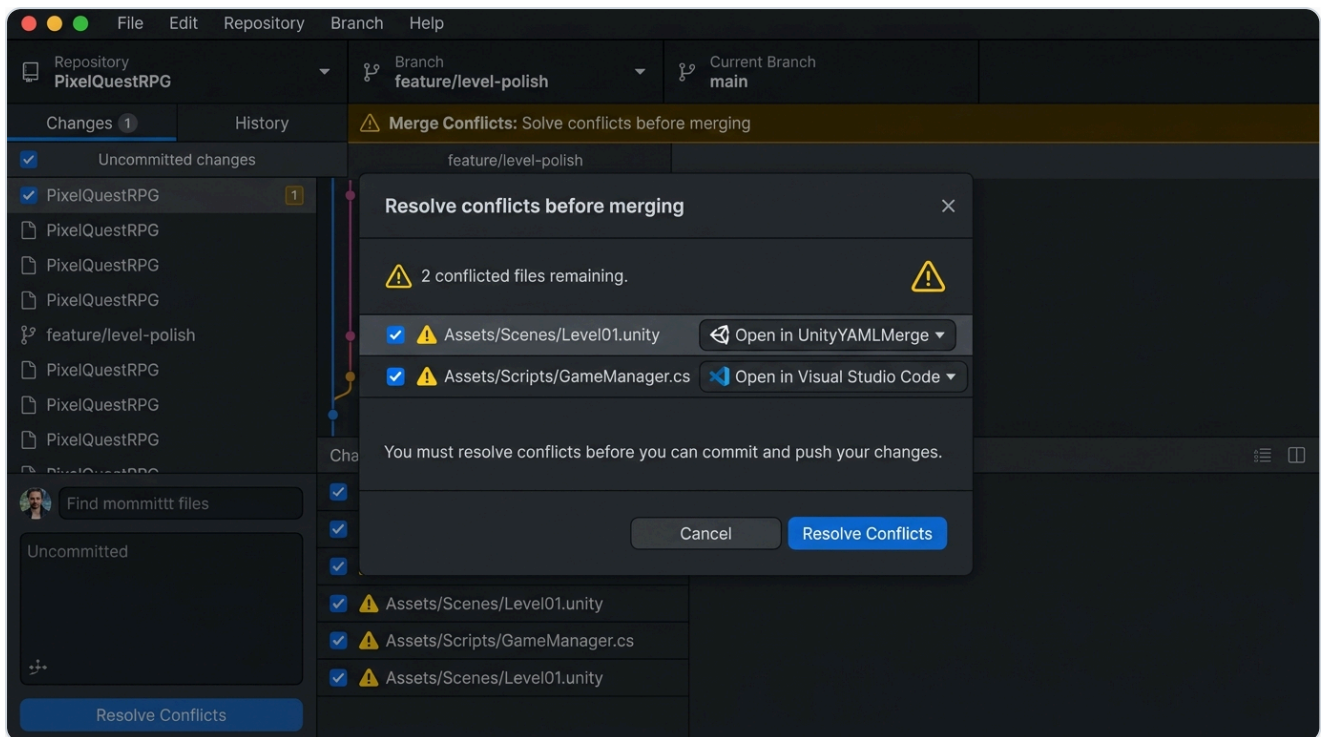


Figura 4.1: Ventana de resolución de conflictos en GitHub Desktop permitiendo derivar la escena a UnityYAMLMerge o elegir versiones completas.

4.5 Tema 5: Forzar una Versión Completa de Asset (--ours vs --theirs)

Modalidad A: Vía Terminal Zsh

```

# Conservar tu versión local
git checkout --ours Assets/Scenes/Nivel01.unity
git add Assets/Scenes/Nivel01.unity
git commit -m "resolve: conservar version local de Nivel01"

# O aceptar completamente la versión remota
git checkout --theirs Assets/Scenes/Nivel01.unity
git add Assets/Scenes/Nivel01.unity
git commit -m "resolve: aceptar version del servidor de Nivel01"

```

Modalidad B: Vía GitHub Desktop

En la ventana modal de conflictos, haz clic en la flecha derecha del archivo y selecciona **Use Modified Version** (tuya) o **Use Existing Version** (remota).

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

```
git stash save "cambios-locales"
git pull --rebase origin main
git stash pop
```

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

```
git fetch origin
git rebase origin/main
git push origin main
```

4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock

Para evitar que dos artistas trabajen sobre el mismo archivo `.psd` o `.blend` en macOS:

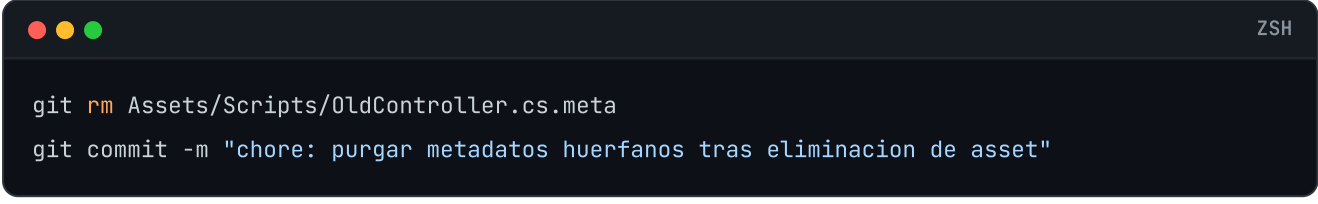
```
# 1. Bloquear archivo antes de editar
git lfs lock Assets/Art/Modelos/JefeFinal.blend

# 2. Consultar bloqueos activos
git lfs locks

# 3. Editar, commitear y subir
git add Assets/Art/Modelos/JefeFinal.blend
git commit -m "art(boss): esculpir detalles de armadura en modelo Blender"
git push origin main

# 4. Desbloquear para el equipo
git lfs unlock Assets/Art/Modelos/JefeFinal.blend
```

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano



```
git rm Assets/Scripts/OldController.cs.meta
git commit -m "chore: purgar metadatos huérfanos tras eliminacion de asset"
```

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en macOS: Múltiples Ramas sin Recargar Library/

```
# Crear worktree paralelo en otra carpeta
git worktree add ../MiJuego-Hotfix hotfix/correccion-audio

# Trabajar en la carpeta secundaria sin tocar el Unity principal
cd ../MiJuego-Hotfix
git push origin hotfix/correccion-audio

# Al terminar, eliminar el worktree
cd ../MiJuegoMac
git worktree remove ../MiJuego-Hotfix
```

5.2 Depuración Binaria con Git Bisect y Blame en C#

```
git bisect start
git bisect bad
git bisect good v1.2.0
# Probar y marcar: git bisect good / git bisect bad
git bisect reset
```

5.3 GitHub Codespaces desde la Terminal de Mac

```
gh codespace create --repo mi-organizacion/MiJuegoMac --branch main
```

5.4 GitHub Copilot CLI en Zsh

```
gh copilot suggest "como revertir un commit de una escena de unity sin tocar scripts"
gh copilot explain "git lfs push --all origin main"
```

5.5 Git Hooks y Validación Pre-commit de Archivos .meta en macOS

Crea el archivo `.git/hooks/pre-commit` :

```
#!/bin/sh
MISSING=0
for f in $(git diff --cached --name-only --diff-filter=A | grep '^Assets/'); do
    if [ "${f##*.}" ≠ "meta" ]; then
        if [ ! -f "${f}.meta" ] && ! git diff --cached --name-only | grep -q "^${f}.meta$";
    then
        echo "❌ ERROR: Falta el archivo .meta para: $f"
        MISSING=1
    fi
fi
done
[ $MISSING -eq 1 ] && exit 1 || exit 0
```

```
chmod +x .git/hooks/pre-commit
```

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes Mediante URLs de Git en UPM

Añade a `Packages/manifest.json` :

```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?
path=src/UniTask/Assets/Plugins/UniTask",
    "com.nevecc.unirx": "https://github.com/nevecc/UniRx.git?path=Assets/Plugins/UniRx"
  }
}
```

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens

Configura en `~/upmconfig.toml` :

```
[npmAuth."https://npm.pkg.github.com/mi-estudio"]
token = "ghp_TU_TOKEN_PERSONAL_CON_SCOPE_READ_PACKAGES"
email = "desarrollador@mi-estudio.com"
alwaysAuth = true
```

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para macOS Standalone (Apple Silicon)

Compila automáticamente aplicaciones de macOS (`.app` y `.dmg`) optimizadas para Apple Silicon en runners de GitHub Actions.

7.2 Activación de Licencias de Unity en GitHub Actions

En **Settings** → **Secrets and variables** → **Actions**, define:

- `UNITY_EMAIL`
 - `UNITY_PASSWORD`
 - `UNITY_LICENSE`
-

7.3 Pipeline Automatizado: Tests y Compilación StandaloneOSX (.app / .dmg)

Crea `.github/workflows/build-macos.yml` :

```
name: Unity macOS Build

on:
  push:
    branches: [ main ]

jobs:
  build:
    name: 🍏 Compilar StandaloneOSX
    runs-on: macos-14
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3
        with:
          path: Library
          key: Library-macOS-${{ hashFiles('Assets/**', 'Packages/**') }}

      - uses: game-ci/unity-builder@v4
        env:
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
        with:
          targetPlatform: StandaloneOSX
          buildName: MiJuegoMac

      - uses: actions/upload-artifact@v4
        with:
          name: Build-macOS-Universal
          path: build/StandaloneOSX
```

7.4 Subida Automática de Builds a GitHub Releases

El pipeline genera un archivo `.zip` o `.dmg` descargable con la aplicación nativa lista para su ejecución en macOS.

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets

Protege la rama `main` en GitHub:

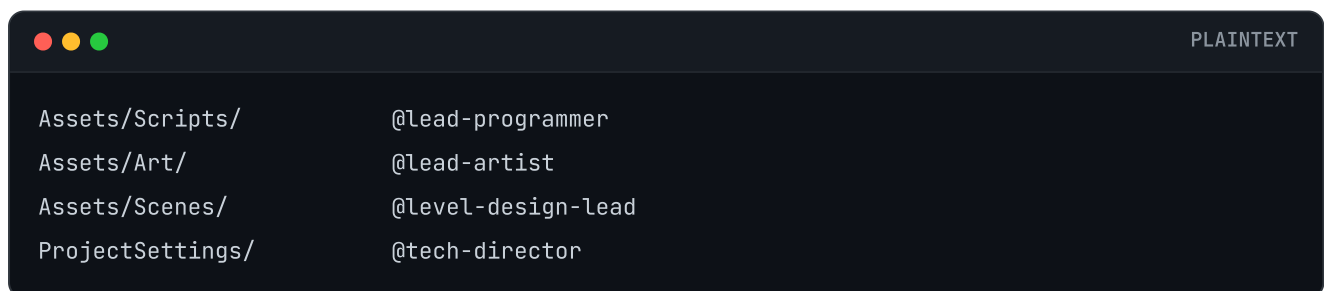
1. Obliga a pasar los tests automáticos de GameCI antes de fusionar.
2. Bloquea pushes directos forzando el uso de Pull Requests.

8.2 Gestión de Secretos para APIs de Juegos (Apple Game Center, Steam)

Guarda las credenciales en variables de entorno o archivos `.env` ignorados por Git. Nunca commitees certificados de desarrollo de Apple o claves de Game Center en scripts C#.

8.3 Gobernanza con CODEOWNERS para Artistas y Programadores

Crea `.github/CODEOWNERS`:

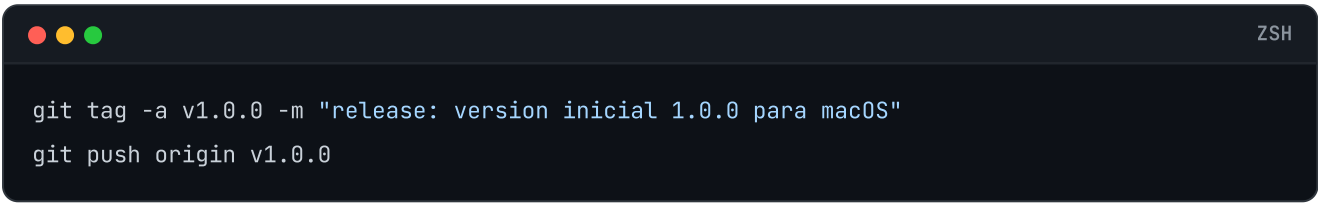


```
Assets/Scripts/      @lead-programmer
Assets/Art/           @lead-artist
Assets/Scenes/        @level-design-lead
ProjectSettings/      @tech-director
```

The screenshot shows a dark-themed code editor window with a title bar containing three colored dots (red, yellow, green) and the word "PLAINTEXT" on the right. The editor displays the content of the `.github/CODEOWNERS` file, which maps specific directory paths to GitHub usernames for review.

Parte IX: Distribución y Despliegue con GitHub Releases

9.1 Creación Automatizada de Releases con Tags Semánticos

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The terminal contains two lines of code:

```
git tag -a v1.0.0 -m "release: version inicial 1.0.0 para macOS"
git push origin v1.0.0
```

9.2 Empaquetado y Publicación de Archivos DMG y Aplicaciones macOS

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text "ZSH" is in the top-right corner. The terminal contains several lines of code, some commented out with #:

```
# Crear archivo comprimido de la aplicación macOS
tar -czf MiJuego-v1.0.0-macOS.tar.gz -C build/StandaloneOSX MiJuegoMac.app

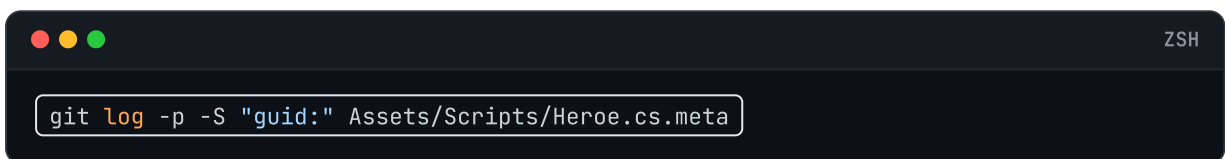
# Publicar el release con GitHub CLI
gh release create v1.0.0 MiJuego-v1.0.0-macOS.tar.gz \
  --title "Mi Videojuego v1.0.0 (macOS Apple Silicon)" \
  --notes "Compilación oficial optimizada para chips Apple Silicon (M1/M2/M3/M4) con backend gráfico Metal."
```

Parte X: Catálogo Maestro de Incidentes Críticos de Unity en macOS

10.1 Incidente 1: "Missing Script" Masivo por Desincronización de GUIDs

- **Solución:**

1. Busca el GUID previo en el historial:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command 'git log -p -S "guid:" Assets/Scripts/Heroe.cs.meta' is entered in the command line.

```
git log -p -S "guid:" Assets/Scripts/Heroe.cs.meta
```

2. Restaura el GUID original en el archivo `.meta` y recarga en Unity (**Assets** → **Reimport All**).

10.2 Incidente 2: Escena Corrupta por Edición Manual o Conflicto Mal Resuelto

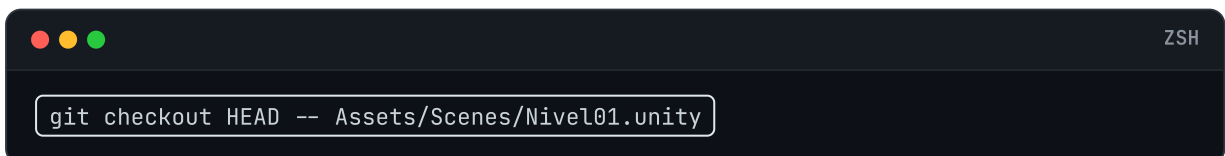
- **Solución:**

1. Busca marcadores residuales de conflicto:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command 'grep -n "<<<<<<" Assets/Scenes/Nivel01.unity' is entered in the command line.

```
grep -n "<<<<<<" Assets/Scenes/Nivel01.unity
```

2. Elimínalos o descarta los cambios locales volviendo a la versión limpia:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The command 'git checkout HEAD -- Assets/Scenes/Nivel01.unity' is entered in the command line.

```
git checkout HEAD -- Assets/Scenes/Nivel01.unity
```

10.3 Incidente 3: Subida Accidental de la Carpeta `Library/` o Archivos

`.DS_Store`

- **Solución:**



```
git rm -r --cached Library/
find . -name ".DS_Store" -exec git rm --cached {} +
git commit -m "fix(git): remover Library y archivos .DS_Store del repositorio"
git push origin main
```

10.4 Incidente 4: Repositorio Bloqueado por Superar el Límite de 100 MB

- Solución con `git-filter-repo` :



```
brew install git-filter-repo
git lfs migrate import --include="*.fbx,*.psd,*.blend,*.wav" --everything
git push origin --force --all
```

10.5 Incidente 5: Shaders Magenta / Rosados tras Clonar en macOS

- **Causa:** Incompatibilidad de pipeline gráfico o falta de compilación de shaders para Metal.
- **Solución:**
 1. Abre **Window** → **Package Manager** y verifica la instalación de `Universal RP` .
 2. Ejecuta en el menú: **Edit** → **Rendering** → **Materials** → **Convert Selected Built-in Materials to URP**.
 3. Purga la caché local de shaders si persiste:



```
rm -rf Library/ShaderCache
```

10.6 Incidente 6: Límite de Almacenamiento y Ancho de Banda de Git LFS Superado

- **Solución:**

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The terminal shows two lines of code: 'git lfs prune --dry-run' and 'git lfs prune', each enclosed in a light blue rounded rectangle.

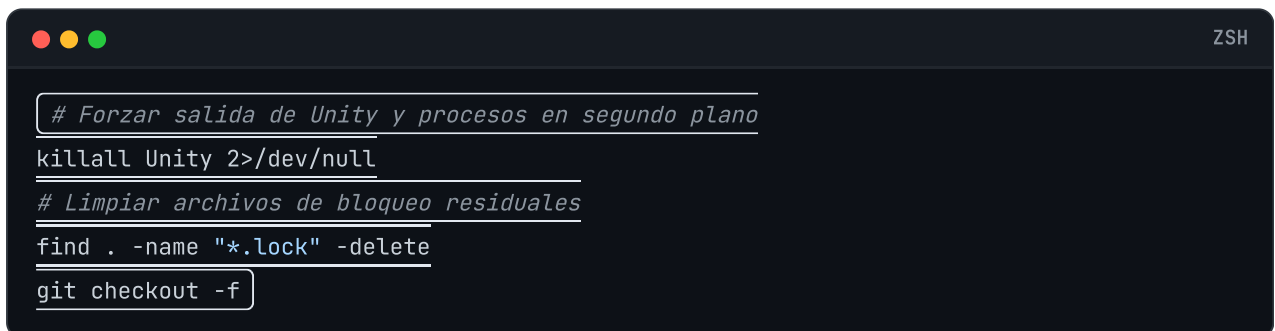
```
git lfs prune --dry-run
git lfs prune
```

10.7 Incidente 7: Desfase de Versiones Menores del Editor de Unity

- **Solución:** Abre siempre el proyecto a través de **Unity Hub**, el cual descargará e iniciará la versión exacta fijada en `ProjectSettings/ProjectVersion.txt`.

10.8 Incidente 8: Archivos Bloqueados por Procesos de Unity en Ejecución

- **Solución:**

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text 'ZSH' is in the top-right corner. The terminal shows several lines of code, some in comments and some in light blue rounded rectangles: '# Forzar salida de Unity y procesos en segundo plano', 'killall Unity 2>/dev/null', '# Limpiar archivos de bloqueo residuales', 'find . -name "*.lock" -delete', and 'git checkout -f'.

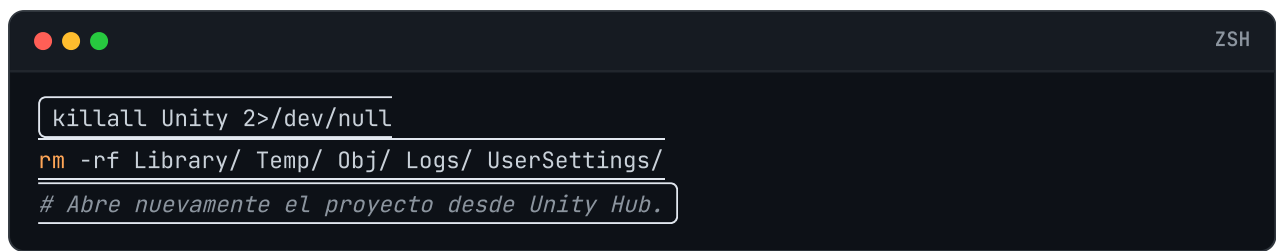
```
# Forzar salida de Unity y procesos en segundo plano
killall Unity 2>/dev/null
# Limpiar archivos de bloqueo residuales
find . -name "*.lock" -delete
git checkout -f
```

10.9 Incidente 9: Fuga de Claves Privadas en ScriptableObjects o Certificados Apple

- **Solución:** Revoca los certificados o API keys inmediatamente y purga el archivo del historial con `git filter-repo`.

10.10 Incidente 10: Regeneración Limpia y Segura de la Caché Local en macOS

- **Solución (El Reset Seguro):**



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar in the top-right corner reads "ZSH". The terminal contains the following text:

```
killall Unity 2>/dev/null  
rm -rf Library/ Temp/ Obj/ Logs/ UserSettings/  
# Abre nuevamente el proyecto desde Unity Hub.
```